

PSEUDOCODE WRITING

**SIMATS**
ENGINEERING

SIMATS Online Programming Lab
JAVA PROGRAMMING

Shasidhar R
192524276RD

QUESTIONS

Q1 ✓
Student Management System [BL3 – Apply]
10 marks

Q2 ✓
Library Book Management [BL3 – Apply]
10 marks

Q3 ✓
Employee Salary Calculation [BL3 – Apply]
10 marks

Q4 ✓
Shape Area Calculator using Polymorphism [BL3 – Apply]
10 marks

Q5 ✓
5. Vehicle Registration System [BL3 – Apply]
10 marks

Q6 ✓
6. Bank Account Hierarchy [BL4 – Analyze]
10 marks

Q7 ✓
7. Hospital Patient Record System [BL4 – Analyze]
10 marks

Q8 ✓
8. E-Commerce Product Catalog [BL4 – Analyze]
10 marks

Q1 Student Management System [BL3 – Apply] 10 marks

Write pseudocode to design a Student class with attributes (name, rollNo, marks) and methods to:

- Accept student details
- Calculate average marks
- Display result (Pass/Fail)

Apply encapsulation and data hiding by restricting direct access to attributes.
OOP Concepts: Encapsulation | Data Hiding | Classes & Objects

Your Answer

```
class Student
private: name, rollNo, marks
public: method acceptDetails()
input name
input rollNo
input marks
end method
method calculateAverage()
average ← marks
return average
end method
method displayResult()
if marks >= 40 then
print "pass"
else
print "fail"
end if
end method
end class
start
create object s1
call s1.acceptDetails() call s1.displayResult()
stop
```

**SIMATS**
ENGINEERING

SIMATS Online Programming Lab
JAVA PROGRAMMING

Shasidhar R
192524276RD

PSEUDOCODE WRITING

← Back

QUESTIONS

Q1 ✓
Student Management System [BL3 – Apply]
10 marks

Q2 ✓
Library Book Management [BL3 – Apply]
10 marks

Q3 ✓
Employee Salary Calculation [BL3 – Apply]
10 marks

Q4 ✓
Shape Area Calculator using Polymorphism [BL3 – Apply]
10 marks

Q5 ✓
5. Vehicle Registration System [BL3 – Apply]
10 marks

Q6 ✓
6. Bank Account Hierarchy [BL4 – Analyze]
10 marks

Q7 ✓
7. Hospital Patient Record System [BL4 – Analyze]
10 marks

Q8 ✓
8. E-Commerce Product Catalog

Q2 Library Book Management [BL3 – Apply] 10 marks

Design a Book class with private attributes (title, author, ISBN, isAvailable). Implement methods to:

- Issue a book (change availability status)
- Return a book
- Display book details using getter methods

Demonstrate object creation for at least 3 books and simulate issue/return operations.
OOP Concepts: Encapsulation | Getters & Setters | Objects

Your Answer

```
class Book
private:
title, author, ISBN, isAvailable
public:
method issueBook()
isAvailable ← false
end method
method returnBook()
isAvailable ← true
end method
method display()
print title, author, ISBN
end method
end class
start
create b1, b2, b3
call b1.issueBook()
call b2.returnBook()
stop
```

PSEUDOCODE WRITING

← Back

QUESTIONS

Q1 ✓
Student Management System
[BL3 – Apply]
10 marks**Q2** ✓
Library Book Management [BL3 – Apply]
10 marks**Q3** ✓
Employee Salary Calculation
[BL3 – Apply]
10 marks**Q4** ✓
Shape Area Calculator using
Polymorphism [BL3 – Apply]
10 marks**Q5** ✓
5. Vehicle Registration System
[BL3 – Apply]
10 marks**Q6** ✓
6. Bank Account Hierarchy [BL4 – Analyze]
10 marks**Q7** ✓
7. Hospital Patient Record
System [BL4 – Analyze]
10 marks**Q8** ✓
8. E-Commerce Product Catalog**Q3 Employee Salary Calculation [BL3 – Apply]**

10 marks

Write pseudocode for an Employee class with encapsulated attributes (empld, name, basicPay, designation). Implement methods to:

- Calculate gross salary (basic + HRA + DA)
- Calculate net salary after deductions (PF, tax)
- Display pay slip

Apply data hiding so salary components are not directly accessible.
OOP Concepts: Encapsulation | Data Hiding | Methods

Your Answer

```
class Employee
private:
    empld,name,basicPay
public:
    method grossSalary()
    gross ← basicPay + 2000
    return gross
end method
    method netSalary()
    net ← grossSalary() - 500
    return net
end method
    method display()
    print empld
    print name
    print grossSalary()
    print netSalary()
end method
end class
start
```

PSEUDOCODE WRITING

← Back

QUESTIONS

Q1 ✓
Student Management System
[BL3 – Apply]
10 marks**Q2** ✓
Library Book Management [BL3 – Apply]
10 marks**Q3** ✓
Employee Salary Calculation
[BL3 – Apply]
10 marks**Q4** ✓
Shape Area Calculator using
Polymorphism [BL3 – Apply]
10 marks**Q5** ✓
5. Vehicle Registration System
[BL3 – Apply]
10 marks**Q6** ✓
6. Bank Account Hierarchy [BL4 – Analyze]
10 marks**Q7** ✓
7. Hospital Patient Record
System [BL4 – Analyze]
10 marks**Q8** ✓
8. E-Commerce Product Catalog**Q4 Shape Area Calculator using Polymorphism [BL3 – Apply]**

10 marks

Create a Shape base class with an overridable calculateArea() method. Apply this to:

- Circle — area using radius
- Rectangle — area using length and breadth
- Triangle — area using base and height

Use method overriding to demonstrate runtime polymorphism. Call calculateArea() for each object.
OOP Concepts: Inheritance | Polymorphism | Method Overriding

Your Answer

```
class shape
method calculatearea
    print area
end method
end class
class circle extends shape
method calculatearea
    area ← 3.14 * r * r
    print area
end method
end class
class Rectangle extends Shape
method calculateArea()
    area ← l * b
    print area
end method
end class
class Triangle extends Shape
method calculateArea()
    area ← 0.5 * base * height
```

PSEUDOCODE WRITING

← Back

QUESTIONS

Q1
Student Management System
[BL3 – Apply]
10 marks**Q2**
Library Book Management [BL3 – Apply]
10 marks**Q3**
Employee Salary Calculation
[BL3 – Apply]
10 marks**Q4**
Shape Area Calculator using
Polymorphism [BL3 – Apply]
10 marks**Q5**
5. Vehicle Registration System
[BL3 – Apply]
10 marks**Q6**
6. Bank Account Hierarchy [BL4 – Analyze]
10 marks**Q7**
7. Hospital Patient Record
System [BL4 – Analyze]
10 marks**Q8**
8. E-Commerce Product Catalog**Q4 Shape Area Calculator using Polymorphism [BL3 – Apply]**

10 marks

Create a Shape base class with an overridable calculateArea() method. Apply this to:

- Circle — area using radius
 - Rectangle — area using length and breadth
 - Triangle — area using base and height
- Use method overriding to demonstrate runtime polymorphism. Call calculateArea() for each object.
OOP Concepts: Inheritance | Polymorphism | Method Overriding

Your Answer

```
print area
end method
end class
class Rectangle extends Shape
method calculateArea()
area ← l * b
print area
end method
end class
class Triangle extends Shape
method calculateArea()
area ← 0.5 * base * height
print area
end method
end class
start
create objects c,r,t
call c.calculateArea()
call r.calculateArea()
call t.calculateArea()
stop
```

PSEUDOCODE WRITING

← Back

QUESTIONS

Q1
Student Management System
[BL3 – Apply]
10 marks**Q2**
Library Book Management [BL3 – Apply]
10 marks**Q3**
Employee Salary Calculation
[BL3 – Apply]
10 marks**Q4**
Shape Area Calculator using
Polymorphism [BL3 – Apply]
10 marks**Q5**
5. Vehicle Registration System
[BL3 – Apply]
10 marks**Q6**
6. Bank Account Hierarchy [BL4 – Analyze]
10 marks**Q7**
7. Hospital Patient Record
System [BL4 – Analyze]
10 marks**Q8**
8. E-Commerce Product Catalog**Q5 5. Vehicle Registration System [BL3 – Apply]**

10 marks

Design a Vehicle class with attributes (regNo, owner, fuelType). Extend it into:

- TwoWheeler — with engine capacity
- FourWheeler — with seating capacity and AC status

Apply inheritance to reuse common attributes and override a displayDetails() method in each subclass.
OOP Concepts: Inheritance | Method Overriding | Subclass

Your Answer

```
class Shape
method calculateArea()
end method
end class
class Circle extends Shape
method calculateArea()
print area
end method
end class
start
create object c
call c.calculateArea()
stop
```

Save Answer

PSEUDOCODE WRITING

← Back

QUESTIONS

Q1
Student Management System
[BL3 – Apply]
10 marks**Q2**
Library Book Management [BL3 – Apply]
10 marks**Q3**
Employee Salary Calculation
[BL3 – Apply]
10 marks**Q4**
Shape Area Calculator using
Polymorphism [BL3 – Apply]
10 marks**Q5**
5. Vehicle Registration System
[BL3 – Apply]
10 marks**Q6**
6. Bank Account Hierarchy [BL4 – Analyze]
10 marks**Q7**
7. Hospital Patient Record
System [BL4 – Analyze]
10 marks**Q8**
8. E-Commerce Product Catalog
10 marks**Q6 6. Bank Account Hierarchy [BL4 – Analyze]**

10 marks

Develop pseudocode for a BankAccount superclass and two subclasses — SavingsAccount (with interest calculation) and CurrentAccount (with overdraft facility). Analyze and implement:

- Inheritance of common attributes (accNo, balance, holder)
- Method overriding for withdrawal behavior
- How overdraft protection differs from savings limit

Compare the behavior of withdraw() across both subclasses and justify the design choices.
OOP Concepts: Inheritance | Method Overriding | Polymorphism

Your Answer

```
class BankAccount
attributes:
accNo,balance,holder
method withdraw()
end method
end class
class SavingsAccount extends BankAccount
method withdraw(amount)
if balance>1000 then balance-=amount
end method
method interest()
print interest
end method
end class
class CurrentAccount extends BankAccount
method withdraw(amount)
balance-=amount
end method
end class
```

QUESTIONS

Q1
Student Management System
[BL3 – Apply]
10 marks**Q2**
Library Book Management [BL3 – Apply]
10 marks**Q3**
Employee Salary Calculation
[BL3 – Apply]
10 marks**Q4**
Shape Area Calculator using
Polymorphism [BL3 – Apply]
10 marks**Q5**
5. Vehicle Registration System
[BL3 – Apply]
10 marks**Q6**
6. Bank Account Hierarchy [BL4 – Analyze]
10 marks**Q7**
7. Hospital Patient Record
System [BL4 – Analyze]
10 marks**Q8**
8. E-Commerce Product Catalog
10 marks**Q7 7. Hospital Patient Record System [BL4 – Analyze]**

10 marks

Design a Patient base class and analyze how it extends into:

- InPatient — with ward number, admission date, and daily charges
- OutPatient — with appointment date and consultation fee

Analyze encapsulation requirements for sensitive data (diagnosis, prescription). Override a generateBill() method for each type and compare the billing logic differences. OOP Concepts: Encapsulation | Inheritance | Polymorphism

Your Answer

```
class Patient
private:
name,id,diagnosis
method generateBill()
end method
end class
class InPatient extends Patient
method generateBill()
bill=daysStayed*dailyCharges
print bill
end method
end class
class OutPatient extends Patient
method generateBill()
print consultationFee
end method
end class
start
create i,o
call i.generateBill()
call o.generateBill()
stop
```

PSEUDOCODE WRITING

← Back

QUESTIONS

- Q1** ✓
Student Management System
[BL3 – Apply]
10 marks
- Q2** ✓
Library Book Management [BL3 – Apply]
10 marks
- Q3** ✓
Employee Salary Calculation
[BL3 – Apply]
10 marks
- Q4** ✓
Shape Area Calculator using
Polymorphism [BL3 – Apply]
10 marks
- Q5** ✓
5. Vehicle Registration System
[BL3 – Apply]
10 marks
- Q6** ✓
6. Bank Account Hierarchy [BL4 – Analyze]
10 marks
- Q7** ✓
7. Hospital Patient Record
System [BL4 – Analyze]
10 marks
- Q8** ✓
8. E-Commerce Product Catalog

Q8 8. E-Commerce Product Catalog [BL4 – Analyze]

10 marks

Analyze and design a Product class hierarchy for an e-commerce system:

- Electronics — with warranty, brand, voltage
 - Clothing — with size, fabric, gender
 - Grocery — with expiryDate and weight
- Override an applyDiscount() method for each category with different discount rules. Analyze why polymorphism makes the cart checkout process flexible and extensible. OOP Concepts: Polymorphism | Inheritance | Method Overriding

Your Answer

```
class Product
method applyDiscount()
end method
end class
class Electronics extends Product
method applyDiscount()
print 10% discount
end method
end class
class Clothing extends Product
method applyDiscount()
print 20% discount
end method
end class
class Grocery extends Product
method applyDiscount()
print 5% discount
end method
end class
start
```

QUESTIONS

- Q1** ✓
Student Management System
[BL3 – Apply]
10 marks
- Q2** ✓
Library Book Management [BL3 – Apply]
10 marks
- Q3** ✓
Employee Salary Calculation
[BL3 – Apply]
10 marks
- Q4** ✓
Shape Area Calculator using
Polymorphism [BL3 – Apply]
10 marks
- Q5** ✓
5. Vehicle Registration System
[BL3 – Apply]
10 marks
- Q6** ✓
6. Bank Account Hierarchy [BL4 – Analyze]
10 marks
- Q7** ✓
7. Hospital Patient Record
System [BL4 – Analyze]
10 marks
- Q8** ✓
8. E-Commerce Product Catalog
[BL4 – Analyze]
10 marks
- Q9** ✓

Q9 9. University Staff Hierarchy [BL4 – Analyze]

10 marks

Analyze the design of a Staff base class extended by TeachingStaff and NonTeachingStaff. Further extend TeachingStaff into Professor and Lecturer (multilevel inheritance). For each level:

- Identify which attributes should be private vs protected
 - Override calculateAllowance() at each level
 - Analyze how protected access enables inheritance without full exposure. Justify the use of multilevel inheritance and explain how access modifiers enforce data hiding.
- OOP Concepts: Multilevel Inheritance | Access Modifiers | Data Hiding

Your Answer

```
class Staff
private:
id,name
protected:
salary
end class
class TeachingStaff extends Staff
method calculateAllowance()
print allowance
end method
end class
class Professor extends TeachingStaff
method calculateAllowance()
print extra
end method
end class
class Lecturer extends TeachingStaff
method calculateAllowance()
print allowance
end method
end class
start
```

QUESTIONS

Q1 ✓
Student Management System
[BL3 – Apply]
10 marks

Q2 ✓
Library Book Management [BL3 – Apply]
10 marks

Q3 ✓
Employee Salary Calculation
[BL3 – Apply]
10 marks

Q4 ✓
Shape Area Calculator using
Polymorphism [BL3 – Apply]
10 marks

Q5 ✓
5. Vehicle Registration System
[BL3 – Apply]
10 marks

Q6 ✓
6. Bank Account Hierarchy [BL4 – Analyze]
10 marks

Q7 ✓
7. Hospital Patient Record
System [BL4 – Analyze]
10 marks

Q8 ✓
8. E-Commerce Product Catalog
[BL4 – Analyze]
10 marks

Q9 ✓

Q10 10. Smart Home Device Controller [BL4 – Analyze]

10 marks

Analyze and design a SmartDevice superclass with subclasses: SmartLight, SmartThermostat, and SmartLock. Override an executeCommand(cmd) method in each, where the same command (e.g., 'on', 'off', 'status') produces device-specific behavior. Analyze:

- How polymorphism allows a single controller loop to manage all devices
- Security implications of data hiding in SmartLock (PIN must never be exposed) Design the class hierarchy and explain the OOP principles applied at each level.

OOP Concepts: Polymorphism | Encapsulation | Inheritance

Your Answer

```
class SmartDevice
private:
    deviceId,status
    method executeCommand(cmd)
end method
end class

class SmartLight extends SmartDevice
    method executeCommand(cmd)
    if cmd="on" then
        print "light on"
    else
        print "light off"
    end if
end method
end class

class SmartThermostat extends SmartDevice
    method executeCommand(cmd)
    print "temperature changed"
end method
end class
```

QUESTIONS

Q1 ✓
Student Management System
[BL3 – Apply]
10 marks

Q2 ✓
Library Book Management [BL3 – Apply]
10 marks

Q3 ✓
Employee Salary Calculation
[BL3 – Apply]
10 marks

Q4 ✓
Shape Area Calculator using
Polymorphism [BL3 – Apply]
10 marks

Q5 ✓
5. Vehicle Registration System
[BL3 – Apply]
10 marks

Q6 ✓
6. Bank Account Hierarchy [BL4 – Analyze]
10 marks

Q7 ✓
7. Hospital Patient Record
System [BL4 – Analyze]
10 marks

Q8 ✓
8. E-Commerce Product Catalog
[BL4 – Analyze]
10 marks

Q10 10. Smart Home Device Controller [BL4 – Analyze]

10 marks

Analyze and design a SmartDevice superclass with subclasses: SmartLight, SmartThermostat, and SmartLock. Override an executeCommand(cmd) method in each, where the same command (e.g., 'on', 'off', 'status') produces device-specific behavior. Analyze:

- How polymorphism allows a single controller loop to manage all devices
- Security implications of data hiding in SmartLock (PIN must never be exposed) Design the class hierarchy and explain the OOP principles applied at each level.

OOP Concepts: Polymorphism | Encapsulation | Inheritance

Your Answer

```
end if
end method
end class
class SmartThermostat extends SmartDevice
    method executeCommand(cmd)
    print "temperature changed"
end method
end class
class SmartLock extends SmartDevice
private:
    PIN
    method executeCommand(cmd)
    print "lock control"
end method
end class
start
create objects l,t,s
call l.executeCommand("on")
call t.executeCommand("status")
call s.executeCommand("off")
stop
```